

PYTHON CONCURRENCY & BACKEND ARCHITECTURE

Async, the GIL, threads vs processes vs queues, and the FastAPI/uvicorn stack — the complete mental model

From async primitives to "why my CPU climbs with traffic" · grounded in a real FastAPI + SQLAlchemy-async + Redis backend · 2026

What this guide is. A ground-up explanation of how concurrency actually works in a Python web backend — the primitives you reach for, when to use threads vs processes vs a task queue, what the GIL really is (explained from the hardware up), how many threads live in a process and why, why CPU rises with request volume even on a non-blocking async server, and where FastAPI / Starlette / uvicorn fit. Boxes: **Why/Pros**, **Cons/Gotchas**, **Key idea**, and **Say it like this** for interview-ready phrasing.

1 · The Mental Model First

async / await gives you **concurrency**, **not parallelism**. One thread, one event loop. When you **await** something that does I/O (DB, Redis, HTTP, SMS), the loop parks that coroutine and runs another. **CPU-bound work does not speed up** — only I/O-bound work does.

Say it like this (the #1 thing interviewers want to hear): "Async overlaps I/O wait time." Never say "async makes code faster" — that fails. A typical backend is I/O-bound (DB + Redis + push), which is exactly why async is the right tool.

2 · The Async Primitives (ranked by how often you use them)

2.1 await — one thing at a time (used constantly)

```
request = await repo.get_by_id(request_id)
await repo.update_status(...)
```

Sequential — each waits for the previous. Use when there's a **data dependency** (you need A's result to do B). Most code is correctly this.

Con: if two awaits are independent, awaiting them serially wastes time (you pay A+B instead of max(A,B)).

2.2 asyncio.gather() — run independent awaitables concurrently

```
location, user_state = await asyncio.gather(
    resolve_location(...), # independent
    resolve_user_state(...), # independent
)
# total time ≈ slower of the two, not the sum
```

When: "I need N independent things to build one response" — dashboards, home feeds, aggregation endpoints. **Frequency: very common.**

Pro: big latency win for independent I/O (3 × 100 ms calls → 100 ms, not 300 ms); returns results in order.

Gotchas interviewers dig into: (1) **return_exceptions** — by default if one task raises, gather propagates that exception but the others keep running detached; set `return_exceptions=True` to get exceptions back as values. (2) **Never gather() coroutines that share the same SQLAlchemy AsyncSession** — a session isn't concurrency-safe; two queries on it at once corrupt its state / throw `InterfaceError`. Give each branch its own session.

```
# each parallel branch gets its OWN session
async with get_async_sessionmaker()() as session:
    ...
```

Say it unprompted: "I use gather for fan-out, but each concurrent branch gets its own DB session — a SQLAlchemy AsyncSession isn't safe to share across concurrent tasks." That one sentence signals real production experience.

2.3 asyncio.create_task() — fire-and-forget / background work

```
asyncio.create_task(track_daily_active_user(client_id, today))
# don't make the user wait for analytics – launch it and move on
```

When: side-effects the response doesn't depend on — analytics, cache warming, a push/SMS.

The bug that bites everyone: (1) the loop keeps only a **weak reference** — if nothing holds the task it can be garbage-collected mid-flight and silently vanish. (2) **Exceptions are swallowed** unless you attach a done-callback. (3) It's **lost on deploy/crash** (lives in web-process memory).

```
# the interview-correct pattern: retain a ref + log errors
_background = set()
def spawn(coro):
    t = asyncio.create_task(coro)
    _background.add(t)
    t.add_done_callback(_background.discard)
    t.add_done_callback(lambda t: t.exception() and log.error(...))
    return t
```

2.4 `asyncio.TaskGroup()` — the modern replacement for `gather` (Python 3.11+)

```
async with asyncio.TaskGroup() as tg:
    t1 = tg.create_task(fetch_a())
    t2 = tg.create_task(fetch_b())
# both done here; if either failed, BOTH are cancelled and errors raised together
```

This is **structured concurrency**: a failure cancels siblings automatically — no orphaned detached tasks like `gather` can leave behind.

Senior signal: "I'd prefer `TaskGroup` over `gather` on 3.11+ because it gives structured concurrency — failures cancel siblings instead of leaking." (`gather` still dominates older codebases.)

2.5 `asyncio.Semaphore` — bounding concurrency

```
sem = asyncio.Semaphore(10)
async def fetch(url):
    async with sem:
        return await client.get(url)
await asyncio.gather(*[fetch(u) for u in thousand_urls])
```

When: fan-out over a large/unbounded list (bulk push sends, bulk notifications). Without the semaphore, `gather` over 1000 items opens 1000 connections at once → you DoS your own DB / get rate-limited.

Pro: protects downstreams. **Con:** adds a tuning knob.

2.6 `asyncio.to_thread()` — escape hatch for blocking/CPU code

```
result = await asyncio.to_thread(blocking_or_cpu_lib, arg)
```

When: you must call a **synchronous** library (image processing, a non-async SDK, `bcrypt`) from async code. It offloads to a thread pool so the event loop stays responsive. This is the answer to "what do you do with a blocking call in async code?"

Con / the deep gotcha: limited by thread-pool size; **not for true CPU parallelism**. It only parallelizes if the blocking call **releases the GIL** (see §4). `Pillow/NumPy/bcrypt` do → real speedup. A pure-Python loop doesn't → no speedup.

2.7 `as_completed` / `wait` — results as they finish / timeouts (niche)

`as_completed` yields results in completion order (good for "show the first result fast"); `wait` with a timeout and `FIRST_COMPLETED` for racing. Worth knowing the names.

2.8 Frequency in real development

Tool	How often you reach for it
<code>await</code> (sequential)	Every handler, all the time
<code>gather</code> / <code>TaskGroup</code>	Common — any endpoint aggregating independent I/O
<code>create_task</code> (fire-and-forget)	Occasional — analytics, push, cache warm
<code>Semaphore</code>	Occasional — bulk/batch fan-out
<code>to_thread</code>	Rare but critical — wrapping a sync lib

as_completed / wait	Rare
---------------------	------

The 80/20: most async code is just `await` in a line. `gather` is the optimization you reach for repeatedly; `create_task` is the one that bites people.

3 · Beyond asyncio — Threads, Processes & Queues

The question is never just "concurrent or not" — it's **two independent axes**: is the work **I/O-bound** (waiting) or **CPU-bound** (burning cycles), and does it run **in-request** or **out-of-request**.

	I/O-bound (waiting on network/disk)	CPU-bound (burning CPU)
In-request	asyncio / threads → great fit	<code>to_thread</code> won't help (GIL) → must use processes
Out-of-request	Queue (if slow) → background task	Celery with process workers → the right answer

3.1 The four tools, what they actually are

- **asyncio** — concurrency, 1 thread, cooperative. I/O-bound only. Lowest overhead. Catch: one blocking call freezes *everything* (the whole server, not one request).
- **Threading** (`threading` , `ThreadPoolExecutor` , `to_thread`) — multiple OS threads, preemptive, but the GIL means only one runs Python at a time. Helps I/O-bound work and calling a blocking lib from async. Does **not** help pure-Python CPU work. **Exception**: C extensions that release the GIL (NumPy, Pillow, bcrypt, lxml) *do* get real parallelism on threads.
- **Multiprocessing** (`multiprocessing` , `ProcessPoolExecutor`) — multiple processes, each with its own interpreter and its own GIL → true multi-core parallelism. Helps CPU-bound pure-Python work. Costs: heavy; data must be **pickled** and copied between processes; slow startup; shared state is hard.
- **Distributed task queue** (Celery / RQ / Dramatiq / Arq) — a separate fleet of worker processes, often on separate machines, fed jobs through a broker (Redis/RabbitMQ). The web server enqueues a job and returns immediately; a worker processes it later. **Not a different way to use CPUs — a different architecture** that moves work out of the request lifecycle entirely.

3.2 When do you reach for a queue (Celery/Arq) instead of threads/to_thread/processes?

Move work to a queue when **any** of these is true:

1. **The work is slow** (seconds–minutes) and the user shouldn't wait. Doing it in-request — even via `to_thread` or a process pool — holds a web worker hostage for the whole duration; under load all workers get tied up and the server stops accepting requests. A queue keeps the web tier snappy (enqueue ≈ 2 ms) while a separate worker tier grinds through it.
2. **The work must survive a crash / be retried (durability)**. `create_task` and FastAPI `BackgroundTasks` live in web-process memory — deploy/crash/OOM and the work is gone silently. A queue persists the job in the broker; a dead worker → the job is redelivered. Retries, dead-letter queues, backoff are built in. If losing the work is a bug (payment email, granting purchased credits), it belongs in a durable queue.
3. **You need to scale the work independently of web traffic**. Heavy report season? Add 10 workers without touching your API servers. You can't do that with in-process threads.
4. **Scheduling / periodic jobs**. Celery Beat runs cron-like jobs (nightly rollups, expiring stale sessions). That's a queue's job, not a request's.

When you do NOT need a queue (keep it in-process): the work is fast (< ~100 ms) and loss-tolerant → `create_task` / `BackgroundTasks`. You just need to not block the loop for a blocking lib → `to_thread`. You need CPU parallelism for one request's result the user waits on → `ProcessPoolExecutor` (rare in web apps).

3.3 The decision tree (memorize this shape)

```
Is the work CPU-bound (burns CPU) or I/O-bound (waits)?
├─ I/O-bound
│   ├── Does the response depend on it?
│   │   ├── YES, multiple independent I/O → asyncio.gather / TaskGroup
│   │   └── YES, single call → just await
│   └── NO (side-effect)
│       ├── fast + loss-tolerant → create_task / BackgroundTasks
│       └── slow OR must-not-lose OR retryable → Celery / queue
└─ CPU-bound
    ├── C-extension that releases the GIL (NumPy/Pillow/bcrypt)?
    │   └── YES → ThreadPoolExecutor / to_thread works
    ├── Pure-Python, fast, response depends on it → ProcessPoolExecutor
    └── Pure-Python, slow, or many of them → Celery with process-pool workers
```

3.4 Worked examples (a fitness-app backend)

Feature	Nature	Right tool	Why
Home feed (location + state + promos)	Multiple independent I/O, user waits	asyncio.gather	Overlap DB/Redis waits
Track daily-active-user	Fast side-effect, loss-OK	create_task (hardened)	Don't block response; analytics loss tolerable
Send OTP SMS	External I/O, must not lose	Queue (or retained task + retry)	A lost OTP = user can't log in
Push to many users (fan-out)	Bulk I/O, slow, retryable	Celery + Semaphore inside	Survives crashes, retries, scales
AI food-scan (~8 s)	Slow external API	Queue → 202 + job_id, push result	Holding a web worker 8 s/scan kills throughput
PDF report generation	CPU + I/O, slow	Celery	Move off the request path entirely
bcrypt password hashing	CPU-bound C ext (releases GIL)	to_thread	Parallelizable, fast enough in-request
Image resize (Pillow)	CPU-bound, releases GIL	to_thread if quick, else Celery	Depends on size/volume
Nightly aggregation	Scheduled	Celery Beat	Cron-style periodic work

The pattern: **fast + tolerant = in-process**; **slow OR durable OR scheduled OR bulk = queue**.

3.5 The tradeoff table

	asyncio	Threads / to_thread	Multiprocessing	Celery / queue
Parallelism	None (1 core)	Only I/O + GIL-releasing C	True multi-core	Multi-core + multi-machine
Best for	I/O-bound	Blocking libs, I/O	CPU-bound pure Python	Slow / durable / scheduled / bulk
Overhead	Tiny	Low	High (spawn + pickle)	Highest (broker + infra)
Survives crash?	No	No	No	Yes (persisted in broker)
Retries/backoff	DIY	DIY	DIY	Built-in
Scales independently	No	No	No (one machine)	Yes
Blocks response?	In request	In request	In request	No (fire & return)

The hidden cost of Celery is operational: now you run a broker, a worker fleet, monitoring (Flower), and you reason about eventual consistency ("the report will be ready soon"). Don't reach for it prematurely.

Senior answer: "Start with BackgroundTasks/create_task; promote to a queue the moment the work becomes slow, must-not-be-lost, or needs to scale separately."

3.6 FastAPI-specific notes

- **FastAPI BackgroundTasks** : runs after the response is sent, but **in the same process** — same fragility as `create_task` (dies on crash/deploy, no retries). Good for loss-tolerant work. It's the natural step between `create_task` and Celery.
- **Modern queue alternatives:** **Dramatiq** (simpler API), **Arq** (async-native, Redis-based — pairs beautifully with an async FastAPI stack), **RQ** (dead simple, Redis). For async FastAPI, **Arq** is often a more natural fit than Celery.

One-liners to internalize: "Threads and async are for I/O; processes and queues are for CPU and durability." · "to_thread only parallelizes if the blocking call releases the GIL — otherwise use a process." · "I move work to a queue when it's slow, must survive a crash, needs retries, or must scale independently — not just because it's 'background.'" · "In-process background work is lost on deploy. If losing it is a bug, it needs a broker."

4 · What the GIL Actually Is (from the hardware up)

4.1 The ladder: from your machine to the GIL

Build it one rung at a time, using a real box: **1 vCPU, 2 GB RAM, 2 uvicorn workers**.

Term	What it physically is	In your world
Core / vCPU	The worker that executes instructions — exactly one thing at a time. Your hard physical ceiling.	You have 1
RAM	Desk space for running programs; caps how much can be in flight (not how many run at once).	2 GB caps your worker count
Process	A running program with its own private memory; the unit the OS schedules & isolates.	Each uvicorn worker = 1 process
CPython	The standard Python interpreter (written in C) — the engine that actually reads & runs your <code>.py</code> .	One copy inside each worker
GIL	A lock inside one CPython: only one thread runs Python bytecode at a time.	1 per worker (so 2 total)
Thread	A line of execution inside a process; threads in one process share memory.	Event loop is one thread
async event loop	One thread juggling many I/O waits cooperatively.	Where your real concurrency comes from

Hierarchy: Machine → CPU core (1 vCPU) → runs Processes (your 2 uvicorn workers) → each has its own CPython + GIL → each can have Threads. You already run 2 processes — that's the real CPU-parallelism lever, because **processes don't share a GIL**.

4.2 What the GIL is, and why it exists

GIL = Global Interpreter Lock — a single mutex inside CPython. The rule: **to execute Python bytecode, a thread must hold the GIL, and there's exactly one GIL per process**. So only one thread runs Python at any instant, no matter how many cores you have.

Why it exists: CPython manages memory with **reference counting** — every object has a counter of how many things point to it; at 0 it's freed. If two threads modified the same refcount on two cores simultaneously, the count would corrupt → memory leak or use-after-free crash. The "proper" fix (a tiny lock per object) would mean millions of lock ops/sec and deadlocks. CPython took a shortcut: **one big lock around the whole interpreter**. Grab the GIL → you're the only thread touching any Python object → refcounting is automatically safe. Simple and fast for single-threaded code; the cost is throwing away multi-core parallelism for Python code.

Key idea: the GIL exists to make memory management thread-safe *cheaply* — not to help you. Its multi-core limitation is a side effect of that design.

4.3 How it works moment-to-moment

```
Thread A: [holds GIL, runs bytecode...] ─┐ releases GIL (every ~5ms, OR when it hits I/O)
Thread B: [waiting...] ─────────────────┘ [grabs GIL, runs bytecode...]
```

A thread holding the GIL gives it up (1) **periodically** (~every 5 ms, so others get a turn — this is why threads feel concurrent), and (2) **when it blocks on I/O** (no Python to run while waiting on the network, so it releases the lock).

4.4 The GIL explains every earlier choice

- **Why threads help I/O but not CPU:** I/O-bound → Thread A blocks on the DB, releases the GIL, Thread B runs → real overlap. CPU-bound → Thread A never blocks, only releases every 5 ms; threads just take turns on one core → no speedup, plus switching overhead.
- **Why asyncio is single-threaded and still fine for I/O:** async doesn't fight the GIL — it's one thread, one loop, overlapping I/O cooperatively. The GIL is irrelevant when there's only one thread.
- **Why `to_thread(pillow_resize)` speeds up but `to_thread(python_loop)` doesn't:** C extensions (Pillow/NumPy/bcrypt) explicitly **release the GIL** while running their C code (it doesn't touch Python refcounts) → real parallelism. A pure-Python loop runs bytecode, which needs the GIL → no parallelism.
- **Why CPU-bound work needs multiprocessing/Celery:** the only way to truly run Python on multiple cores is multiple processes — each has its own interpreter and its own GIL. No shared lock = real parallelism.

One-paragraph interview answer: "The GIL is a single lock in CPython that lets only one thread execute Python bytecode at a time. It exists to make refcount-based memory management thread-safe cheaply. The consequence: Python threads don't give multi-core parallelism for CPU-bound work — they only help for I/O, because a thread releases the GIL while waiting on I/O (and C extensions can release it during their own work). For real CPU parallelism you need multiple processes, each with its own GIL — though CPython is now working toward an optional no-GIL build (PEP 703, 3.13)."

Note: the GIL is a **CPython** thing, not a Python-language thing — Jython/IronPython have none. Python 3.13 ships an experimental free-threaded (no-GIL) build.

5 · How Many Threads Live in One Process?

You don't pick a fixed number, and it's not just "1 or 2." For a pure async FastAPI app inside one uvicorn worker:

1. **The main thread** runs the **event loop** — all your `async def` code runs here, on this one thread, juggling hundreds of requests by overlapping I/O. This is why we say "single thread."
2. **A background thread pool**, created on demand (idle until needed), grows up to a cap. Used by `to_thread / run_in_executor` and by sync (`def`) routes.
3. **A few odds and ends** — some drivers / monitoring agents spawn 0–3 threads.

State of one worker	Threads (roughly)
Pure async app, idle	~1–3 (event loop + maybe driver/monitor)
Async app actively using <code>to_thread</code>	up to ~5 more (asyncio pool cap on 1 vCPU)
If you have <code>def</code> (sync) routes under load	up to ~40 more (AnyIO pool)

Key idea: threads here are **not** "how many requests I handle" — the event loop handles hundreds of concurrent requests on ONE thread. The extra threads only exist to babysit blocking work that would otherwise freeze that one precious event-loop thread.

5.1 Where the two numbers (5 and 40) come from — derived differently

Pool	Number	Fixed or scales?	Source
asyncio <code>to_thread</code> pool	<code>min(32, cpu_count + 4)</code> → 5 on 1 vCPU	Scales with CPU , capped at 32	CPython <code>ThreadPoolExecutor</code>
AnyIO sync-route pool	40	Fixed constant (same on 1 or 64 vCPU)	AnyIO <code>CapacityLimiter</code>

The asyncio formula `min(32, cpu+4)` : **+4** so even a 1-core box can overlap several blocking I/O calls; **cap 32** because threads cost memory + context-switching and this pool is for I/O-ish offloading. AnyIO's **40** is just a number its authors picked as "a reasonable max concurrent sync requests."

Both are overridable:

```
# asyncio pool
loop.set_default_executor(ThreadPoolExecutor(max_workers=10))
# AnyIO / sync-route pool
anyio.to_thread.current_default_thread_limiter().total_tokens = 80
```

The real bottleneck to watch: you almost never touch the asyncio one, but AnyIO's **40** can throttle an app with lots of `def` routes doing blocking DB calls — they queue once 40 are in flight.

Count them on your box: `ps -o nlwp= -p <PID>` (NLWP = thread count) or in Python `threading.active_count()`.

6 · Why CPU Climbs With Request Volume (even on a non-blocking async server)

The misconception: "async = non-blocking = free, so CPU shouldn't move." Wrong. **Every request has two parts:**

```
A single request = [ WAITING on I/O ] + [ ACTUALLY COMPUTING ]
                   (DB, Redis, push)   (parsing, validating, serializing)
                   ← async overlaps this → ← async does NOTHING for this →
```

- **Part A — waiting:** while the DB thinks, your code does nothing; the loop runs another request. Overlapped for free.
- **Part B — computing:** the actual CPU work each request needs. Still has to happen, burns CPU, and runs single-file on the one event-loop thread. Async does **not** make it disappear or overlap.

CPU usage = the sum of all the Part-B work. More requests → more Part B → more CPU.

6.1 What "Part B" really is (bigger than you think)

For one `/home` request the event-loop thread actually executes: HTTP parse + headers, TLS decrypt/encrypt, JSON-decode the body, **Pydantic validation** (every field checked), your business logic/math, **hydrating ORM rows** into Python objects, and **JSON-encoding the response** (surprisingly expensive in Python). Say that's ~5 ms of CPU. The DB wait (50 ms) is overlapped; the 5 ms of CPU is not.

6.2 The ceiling this creates (the "aha")

```
One event loop = one thread = one core. If each request needs 5ms CPU:
1000ms / 5ms = ~200 requests/sec of CPU work — no matter how good async is.
```

```
Low traffic: CPU 10% — loop mostly idle, waiting on I/O.
Rising:      CPU 40% — more Part-B work piling up.
High:        CPU 95% — loop barely keeps up.
Past the knee: CPU 100% — requests QUEUE even though nothing is "blocked".
               Latency spikes; async can't save you — you're out of CPU.
```

Key idea: once you saturate the core's CPU, "non-blocking" stops helping — tasks are *ready* to run but there's no CPU free to run them, so they wait their turn.

6.3 Why adding CPU (vCPUs) fixes it, not RAM

RAM grows with concurrency (each in-flight request holds buffers/objects) and caps how many connections you can hold — but you usually hit the **CPU wall first**. More vCPUs let you run more workers (more event loops) in true parallel → more total Part-B throughput. This is exactly *why* you run multiple uvicorn workers: each worker = its own event loop = its own core = its own GIL.

```
1 vCPU, 2 workers → ~2 event loops time-slicing 1 core (limited)
4 vCPU, 4 workers → 4 event loops on 4 cores → ~4x CPU throughput
```

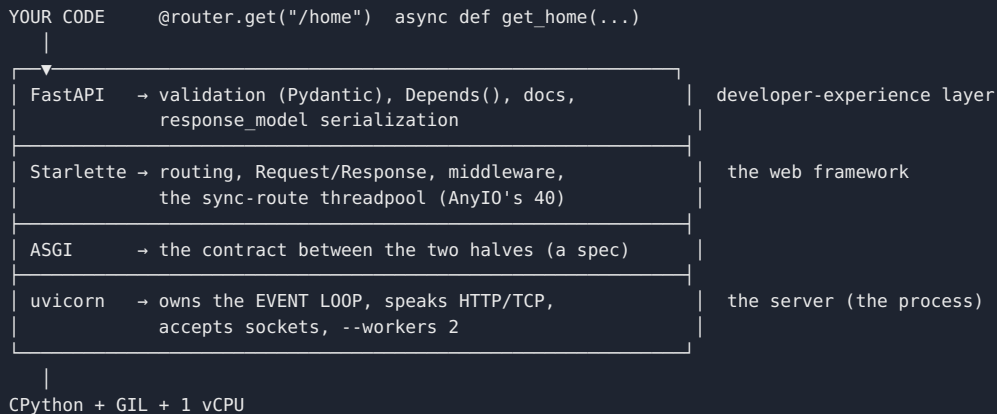
Corrected mental model: "Async doesn't make my server do less work — it stops it sitting idle during I/O waits. Every request still costs CPU to parse, validate, and serialize, and that runs single-file on one event-loop thread. So request volume drives CPU up; when one core saturates I add workers/vCPUs to get more cores doing that CPU work in parallel."

Senior corollary (ties it all together): CPU climbing from many *small* per-request costs (JSON, validation, ORM) → add workers/vCPUs (scale horizontally). CPU climbing from a *few heavy* ops (image, AI, reports) → don't scale the web tier; move that work to a queue, or it eats the same event-loop CPU that should serve 200 light requests.

Rule of thumb: sync apps ≈ $(\text{vCPUs} \times 2) + 1$ workers; async apps often need fewer (each worker already handles tons of concurrent I/O). On 1 vCPU, 2 workers is reasonable.

7 · Where FastAPI / Starlette / uvicorn / ASGI Fit

So far we described the engine (event loop, threads, cores) but not who turns an HTTP request into your Python function. The software layer-cake:



7.1 Who does what

- **uvicorn = the server. It owns the event loop.** The event loop we kept discussing is created and run by uvicorn. `--workers 2` is uvicorn forking into 2 processes, each running one event loop. It listens on the socket, speaks HTTP (bytes ↔ request/response), and calls your app per request. You run `uvicorn main:app` — uvicorn is the runner, `app` is the thing being run.
- **Starlette = the web framework** under FastAPI: routing (URL → function), Request/Response objects, middleware (auth/CORS/logging), and the **sync-route threadpool** (the AnyIO 40 — how `def` routes run off the event loop). FastAPI subclasses Starlette.
- **FastAPI = the developer-experience layer** on top of Starlette: Pydantic validation, dependency injection (`Depends(...)`), automatic JSON serialization of your return value, and auto OpenAPI/Swagger docs. None of these exist in Starlette.
- **ASGI = the contract** (a simple async function signature) between the server (uvicorn) and the app (FastAPI/Starlette). Just a spec, not code you run.

7.2 One real request, traced through the layers

```
1. uvicorn - bytes arrive; decrypt TLS, parse HTTP, build ASGI scope, call app on the loop. [server]
2. Starlette - match path /home → get_home(); run middleware (auth, logging). [routing]
3. FastAPI - resolve Depends(): open DB session, verify JWT, validate query params. [validation/DI] ← CPU
4. YOUR CODE - get_home() runs ON THE EVENT LOOP; await DB/Redis (Part A overlapped, Part B = CPU).
5. FastAPI - serialize the returned object to JSON via response_model. [serialization] ← CPU
6. Starlette - wrap it in a Response.
7. uvicorn - encode HTTP, TLS-encrypt, write to socket. [server]
```

Tie-back: steps 3 & 5 (FastAPI's validation + serialization) are pure Part-B CPU — a big chunk of "why CPU rises with traffic." FastAPI's convenience costs CPU on the event-loop thread. Worth it, but not free.

One-sentence summary: "uvicorn is the server that owns the event loop and speaks HTTP; Starlette routes a request to a function; FastAPI validates inputs, injects dependencies, and serializes outputs. My code is just a function FastAPI hands a clean request to and takes a typed response from."

8 · Interview Question Bank (what gets asked, in order of likelihood)

Question	The answer in one line
Concurrency vs parallelism / why is async faster?	It overlaps I/O wait time. It does NOT help CPU-bound work.
<code>gather</code> vs <code>create_task</code> vs <code>await</code> ?	all-and-wait / fire-and-forget / sequential.
3 independent API calls in a handler — speed it up?	<code>gather</code> / <code>TaskGroup</code> .
What if one task in <code>gather</code> raises?	Propagates; siblings keep running unless cancelled; <code>return_exceptions=True</code> changes it.
Blocking/sync function — call it from async?	<code>asyncio.to_thread</code> / run in executor.
Limit concurrency over 10,000 tasks?	<code>Semaphore</code> .
Why can't you share a DB session across gathered tasks?	An <code>AsyncSession</code> isn't concurrency-safe; one session = one logical transaction.
Does <code>time.sleep(5)</code> work in async?	No — it blocks the whole loop. Use <code>await asyncio.sleep(5)</code> . (Classic trap.)
What is structured concurrency / <code>TaskGroup</code> ?	Failures cancel siblings; no leaked detached tasks — senior differentiator.
What is the GIL?	One CPython lock; only one thread runs Python bytecode at a time; exists for cheap refcount safety.
Slow 8-second operation in a request — architect it?	Don't keep it in-request — enqueue, return 202 + <code>job_id</code> , deliver via polling/WebSocket. (Durability + scaling.)
When is reaching for Celery overkill?	Fast + loss-tolerant work — the broker/worker/monitoring overhead has no payoff; use <code>create_task</code> .

8.1 The two scars worth volunteering unprompted

1. "Async only helps I/O-bound workloads; for CPU-bound I'd use multiprocessing or offload a GIL-releasing call with `to_thread`."
2. "When I fan out with `gather`, each concurrent branch gets its own DB session — a SQLAlchemy `AsyncSession` isn't safe to share across concurrent tasks." (A real production scar — the most convincing kind of answer.)

8.2 The biggest gap most engineers have

The jump from "concurrency primitive" to "system architecture." When work is slow, the weak instinct is "how do I not block the loop" (`to_thread`); the strong instinct is "this shouldn't be in the request at all" → queue with a `job-id` + async result delivery. Drill the trigger: **slow / must-not-lose / bulk / scheduled** → **it leaves the request and goes to a queue.**

9 · Bonus — Production Patterns From a Real Code Review

Two systemic correctness issues that recur in event-driven async backends, and the fixes:

9.1 Fire events / fan-out / cache-invalidation AFTER commit, not before

The bug: publishing domain events, pushing WebSocket messages, or invalidating caches *before* `await db.commit()`. Three concrete failures: (1) a notification handler opens its own session and reads state on a different connection before your transaction committed → misses the row you just wrote; (2) WebSocket recipients see a message before the sender's transaction commits — if commit fails, they saw something that doesn't exist; (3) cache invalidation racing a concurrent read repopulates from *pre-commit* state.

The fix (clean): an **after-commit hook** — buffer domain events and flush them in the route *after* `await db.commit()` (or a SQLAlchemy `after_commit` listener / transactional outbox). With an event-bus abstraction this is a localized change, not a rewrite.

9.2 Read-modify-write races on state transitions

The bug: `get_by_id` → `object.accept()` → `update_status` where the UPDATE has no guard (`SET status='accepted'` with no `WHERE status='pending'`). Two concurrent accepts both read PENDING and both win. Same risk in any "accept/join/claim" flow and in webhook payment idempotency done in Python only.

The fix: put the predicate in the UPDATE — `UPDATE ... SET status='accepted' WHERE id=? AND status='pending'` — and check `rowcount` (0 rows = someone beat you). Or `SELECT ... FOR UPDATE` (pessimistic lock). For payments, enforce idempotency with a unique constraint on the idempotency key at the DB level, not just an in-Python check.

9.3 Things this codebase did genuinely well (worth emulating)

- **Hexagonal / ports-and-adapters:** a Protocol interface + a `build_*_api()` factory doing DI, with a `NoopEventBus()` null-object default. A real domain layer with frozen value objects, invariant enforcement in `__post_init__`, and typed state-machine errors.
- **Resilience instincts:** a `_safe_redis` helper with a 50 ms timeout + circuit breaker + graceful fallback, so Redis going down degrades the feed instead of taking it down.
- **Security by construction:** `client_id` always from the verified JWT, never the request body (IDOR-safe); refresh-token rotation with reuse-detection; no raw/f-string SQL; authorization enforced in the domain layer.